

Mathematical Functions

Mathematical Functions

Plant Simulation provides a number of **mathematical functions** .

You do not have to enter a path for the **mathematical functions**.

- [Basic Mathematical Functions](#)
- [Functions for Numerical Values](#)
- [Trigonometric Functions](#)
- [Distribution Functions, z_ functions](#)

See also

[Arithmetic Operators](#)

Basic Mathematical Functions

Plant Simulation provides a number of **basic mathematical functions**.

You can specify these **basic mathematical functions**.

Function	Type	Results in
abs [SimTalk]	<code>abs(x)</code>	the absolute value of $ x $
beta [SimTalk]	<code>beta(x, y)</code>	the beta function $\text{beta}(x, y)$, $x, y > 0$
betai [SimTalk]	<code>betai(x, y, z)</code>	the incomplete beta function $\text{betai}(x, y)$, $x, y > 0$ and $0 \leq z \leq 1$
exp [SimTalk]	<code>exp(x)</code>	the exponential function e^x The parameter is any number of data type <i>real</i> .
gamma [SimTalk]	<code>gamma(x)</code>	the gamma function $\Gamma(x)$, $x > 0$
log [SimTalk]	<code>log(x)</code>	the natural logarithm
log10 [SimTalk]	<code>log10(x)</code>	the logarithm to the base of 10
pow [SimTalk]	<code>pow(x, y)</code>	x^y If the basis is 0, the exponent has to be greater than 0. If the exponent is not an integer, for example -0.5, the basis has to be greater than or equal to 0.
exponentiation operator [SimTalk]	<code>x ** y</code>	the exponentiation operator. Is the same as <code>pow(x, y)</code> .
sqrt [SimTalk]	<code>sqrt(x)</code>	the square root. The parameter of data type <i>real</i> has to be equal to or greater than 0.

* The square root function *sqrt* only accepts parameters greater than or equal to 0.

* The functions *sqrt* and *pow* also support the use of a physical unit. For *sqrt* this is limited though. If the operand has a square unit, the result has the non-square unit. For other powers of units *Plant Simulation* returns a number of data type *real* without a unit.

Note

When using the functions *exp*, *pow*, *gamma* and the arithmetic operators in *SimTalk*, you have to make sure that the result is located within the range of numbers your computer can show. Use the gamma function to compute the factorial of a number n : $n! = 1 \cdot 2 \cdot \dots \cdot n = \Gamma(n+1)$

Parameter

The parameter for the **basic mathematical functions** has the data type *real* or *integer* or it can be a value with a physical unit.

Return Value

The return value has the data type *real*.

Example

```
var area := 2m * 2m;  
print area      // outputs 4m² in the Console  
print sqrt(area) // outputs 2m in the Console
```

```
-- exponentiation operator ** instead of pow  
var x: real := 2  
var y: real := 3  
var z: real := x ** y // z = 2*2*2 = 8
```

See also

Arithmetic Operators

Functions for Numerical Values

Plant Simulation provides a number of **functions for numerical values**.

You can specify these **functions for numerical values**.

Function	Type	Results in
ceil [SimTalk]	<code>ceil(x)</code>	the smallest integer number greater than or equal to x
floor [SimTalk]	<code>floor(x)</code>	the greatest integer number smaller than or equal to x
max [SimTalk]	<code>max(x,y[,z, ...])</code>	the maximum
min [SimTalk]	<code>min(x,y[,z, ...])</code>	the minimum
round [SimTalk]	<code>round(x)</code>	the next integer number. <code>print round(-1.5)</code> yields -2
round [SimTalk]	<code>round(x:real, places:integer)</code>	the rounding of the real number x to the number of specified places

Note

You can specify more than two parameters for the functions *max* and *min*.

For the functions *max* and *min* you cannot mix data types arbitrarily, i.e., all data types either have to be numerical, or all data types have to be of data type *date* and *dateTime*, or all have to be of data type *string*.

Parameters

The parameter for the **functions for numerical values** can have the data type *real*, *integer*, or it can be a value with a physical unit.

The parameter can also have the data type *date*, *dateTime*, or *string* for the functions *min* and *max*.

Return Value

The return value has the same data type as the parameter that you specify.

See also

Arithmetic Operators

Trigonometric Functions

Trigonometric Functions

Plant Simulation provides a number of **trigonometric functions**.

You can specify these **trigonometric functions**.

Function	Type	Results in
acos [SimTalk]	<code>acos(x)</code>	the arc cosine of x
asin [SimTalk]	<code>asin(x)</code>	the arc sine of x
atan [SimTalk]	<code>atan(x)</code>	the arc tangent of x
atan2 [SimTalk]	<code>atan2(y, x)</code>	the arc tangent of y and x
cos [SimTalk]	<code>cos(x)</code>	the cosine of x
sin [SimTalk]	<code>sin(x)</code>	the sine of x
tan [SimTalk]	<code>tan(x)</code>	the tangent of x

Parameter

Trigonometric functions expect the parameter in radians or return the result in radians. You can specify parameters of data type *real* or *integer*.

Return Value

The return value has the data type *real*.

See also

[atan2 \[SimTalk\]](#)

atan2 [SimTalk]

The function `atan2(y, x)` returns the angle in radians between the positive x-axis of a plane and the point given by the coordinates (x, y) on it for any real number arguments x and y, both of which are not equal to zero.

Type

Function

Syntax

```
atan2(y:real, x:real) → real
```

Parameters

You can specify the following parameters:

- The parameter *y* of data type *real* designates the coordinate on the y-axis.
- The parameter *x* of data type *real* designates the coordinate on the x-axis.

Return Value

The return value has the data type *real*.

The angle is positive for counter-clockwise angles (upper half-plane, $y > 0$), and negative for clockwise angles (lower half-plane, $y < 0$).

For $x=0$ and $y=0$ the function returns 0.

Back to

Trigonometric Functions

Distribution Functions

Distribution Functions, *z_* functions

Create probability distributions with the distribution functions (*z_* functions).

Remarks

You can create random numbers for times in the dialogs of the objects on the **Tab Times** and on the **Tab Failures**, with a **Variable**  of data type *randtime*, and in *SimTalk* with the functions described below.

Plant Simulation provides these **distribution functions**.

Function	Results in this distribution
<code>z_beta([Stream:integer,]Alpha1:real,Alpha2:real) -> real</code>	Beta [distribution] , z_beta [SimTalk]
<code>z_binomial([Stream:integer,]n:integer,p:real) -> real</code>	Binomial [distribution] , z_binomial [SimTalk]

Function	Results in this distribution
<code>z_cauchy([Stream:integer,]Mu:time,Theta:real[,LowerBound:real,UpperBound:real]) -> real</code>	Cauchy [distribution] , z_cauchy [SimTalk]
<code>z_cemp([Stream:integer,]Table:table) -> real</code>	cEmp [continuous empirical distribution] , z_cEmp [SimTalk]
<code>z_demp([Stream:integer,]Table:table) -> real</code>	dEmp [discrete empirical distribution] , z_dEmp [SimTalk]
<code>z_emp([Stream:integer,]Table:table,Column:integer) -> integer</code>	Emp [primitive empirical distribution] , z_emp [SimTalk]
<code>z_erlang([Stream:integer,]Mu:time,Sigma:time[,LowerBound:real,UpperBound:real]) -> real</code>	Erlang [distribution] , z_erlang [SimTalk]
<code>z_frechet([Stream:integer,]Mu:time,Theta:real[,LowerBound:real,UpperBound:real]) -> real</code>	Fréchet [distribution] , z_frechet [SimTalk]
<code>z_gamma([Stream:integer,]Alpha:real,Beta:time[,LowerBound:real,UpperBound:real]) -> real</code>	Gamma [distribution] , z_gamma [SimTalk]
<code>z_geom([Stream:integer,]p:real[,LowerBound:real,UpperBound:real]) -> real</code>	Geom [distribution] , z_geom [SimTalk]
<code>z_gumbel([Stream:integer,]Mu:time,Theta:real[,LowerBound:real,UpperBound:real]) -> real</code>	Gumbel [distribution] , z_gumbel [SimTalk]
<code>z_hypgeom([Stream:integer,]m:integer,n:integer,p:real) -> real</code>	Hypgeo [distribution] , z_hypgeom [SimTalk]
<code>z_laplace([Stream:integer,]Mu:time,Theta:real[,LowerBound:real,UpperBound:real]) -> real</code>	Laplace [distribution] , z_laplace [SimTalk]
<code>z_logistic([Stream:integer,]Mu:time,Theta:real[,LowerBound:real,UpperBound:real]) -> real</code>	Logistic [distribution] , z_logistic [SimTalk]
<code>z_loglogistic([Stream:integer,]Mu:time,Theta:real[,LowerBound:real,UpperBound:real]) -> real</code>	Loglogistic [distribution] , z_loglogistic [SimTalk]
<code>z_lognorm([Stream:integer,]Mu:time,Sigma:time[,LowerBound:real,UpperBound:real]) -> real</code>	Lognorm [distribution] , z_lognorm [SimTalk]

Function	Results in this distribution
<code>z_negexp([Stream:integer,]Beta:time[, LowerBound:real,UpperBound:real]) -> real</code>	Negexp [distribution] , z_negexp [SimTalk]
<code>z_normal([Stream:integer,]Mu:time,Sigma:time[, LowerBound:real,UpperBound:real]) -> real</code>	Normal [distribution] , z_normal [SimTalk]
<code>z_paralogistic([Stream:integer,]Mu:time,Theta:real[, LowerBound:real,UpperBound:real]) -> real</code>	Paralogistic [distribution] , z_paralogistic [SimTalk]
<code>z_pareto([Stream:integer,]Mu:time,Theta:real[, LowerBound:real,UpperBound:real]) -> real</code>	Pareto [distribution] , z_pareto [SimTalk]
<code>z_poisson([Stream:integer,]Lambda:real) -> real</code>	Poisson
<code>z_triangle([Stream:integer,]c:real,a:real,b:real) -> real</code>	Triangle [distribution] , z_triangle [SimTalk] [triangular distribution]
<code>z_uniform([Stream:integer,]LowerBound:real,UpperBound:real) -> real</code>	Uniform [distribution] , z_uniform [SimTalk]
<code>z_weibull([Stream:integer,]Alpha:real,Beta:time) -> real</code>	Weibull [distribution] , z_weibull [SimTalk]

Parameters

You can specify the following parameters:

- The parameter *Stream* of data type *integer* designates the random number stream (compare [Random Number Seed Values](#)).

If you call one of the distribution functions in a *Method*, the parameter *Stream* is optional. If you do not specify the stream, *Plant Simulation* uses the random number stream of the *Method* or of the *user-defined attribute* of data type *method*.

You can also call the distribution functions with the parameter *Stream*. For formulas this is required, as formulas never use the random number stream of the surrounding object.

- All other parameters are the parameters of the corresponding distribution function as described under [Probability Distributions](#). They all either are of data type *real* or *integer*.

Return Value

The return value has the data type *real*.

The distribution function `z_emp` returns a value of data type *integer*.

SimTalk

[RandomSeed \[SimTalk\] - Method/User-defined Attribute](#)

See also

[resetRandomNumberStream \[SimTalk\]](#)

[Parameters of the Distributions](#)

[Probability Distributions](#)

[Methods of the Distributions](#)

Cauchy [distribution], z_cauchy [SimTalk]

The *Cauchy distribution* is a continuous probability distribution.

Remarks

Use the *Cauchy distribution* to represent the occurrence of failures while developing large systems.

Syntax

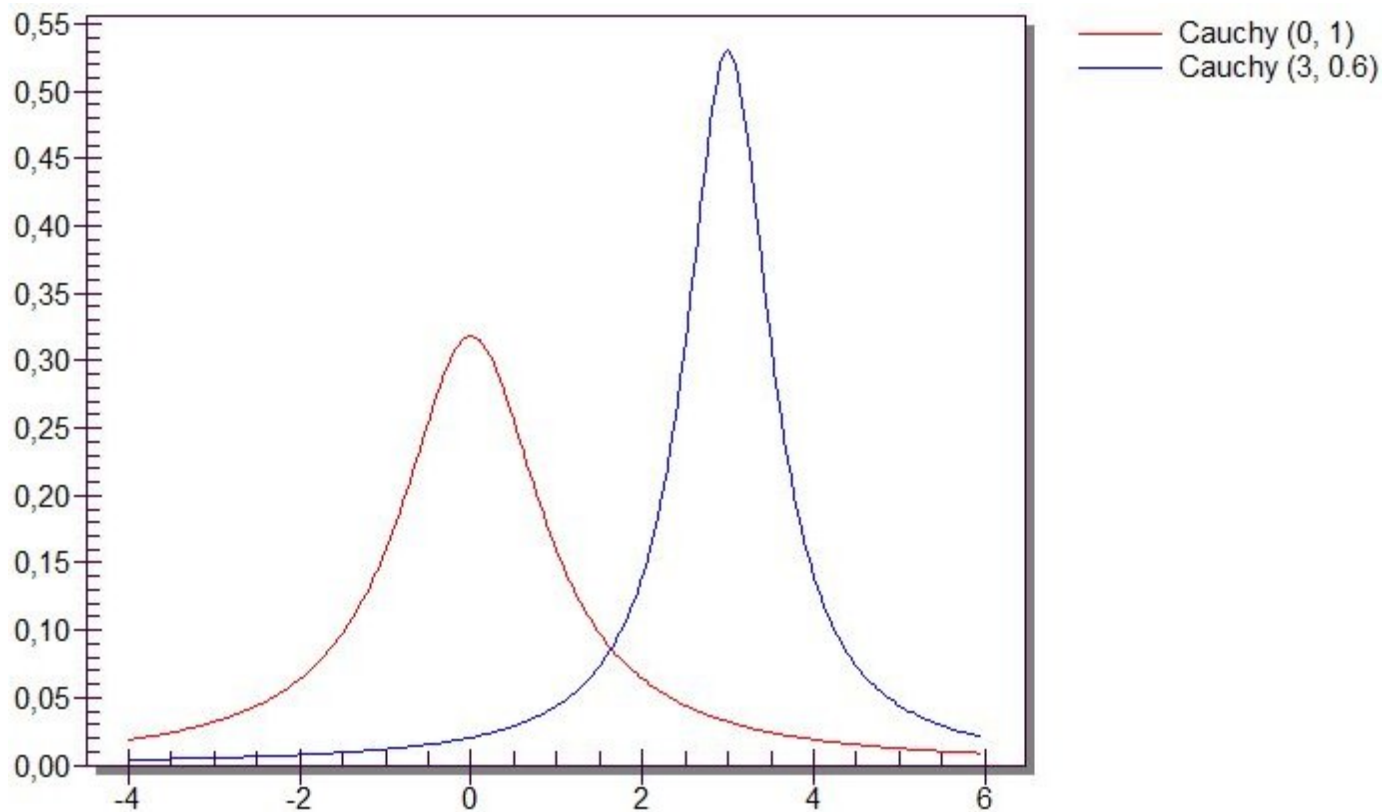
```
z_cauchy([Stream:integer,]Mu:time,Theta:real[,LowerBound:real,UpperBound:real]) → real
```

Parameters

The parameters are μ and θ both of which have to be greater than 0. The mean value and the standard deviation cannot be computed from the parameters.

The density function is
$$f(x) = \frac{\theta}{\pi (\theta^2 + (x - \mu)^2)}$$

The distribution function is
$$F(x) = \frac{1}{2} + \frac{1}{\pi} \cdot \text{atan} \left(\frac{x - \mu}{\theta} \right)$$



Return Value

The return value has the data type *real*.

SimTalk

Trigonometric Functions

See also

Parameters of the Distributions

Fréchet [distribution], z_frechet [SimTalk]

The *Fréchet distribution* is also called the *Gumbel type II distribution* or the *inverse Weibull distribution*.

Remarks

Use the *Fréchet distribution* and the *Gumbel distribution* if you want to find out the maximum of several random numbers.

Syntax

```
z_frechet([Stream:integer,]Mu:time,Theta:real[,LowerBound:real,UpperBound:real]) → real
```

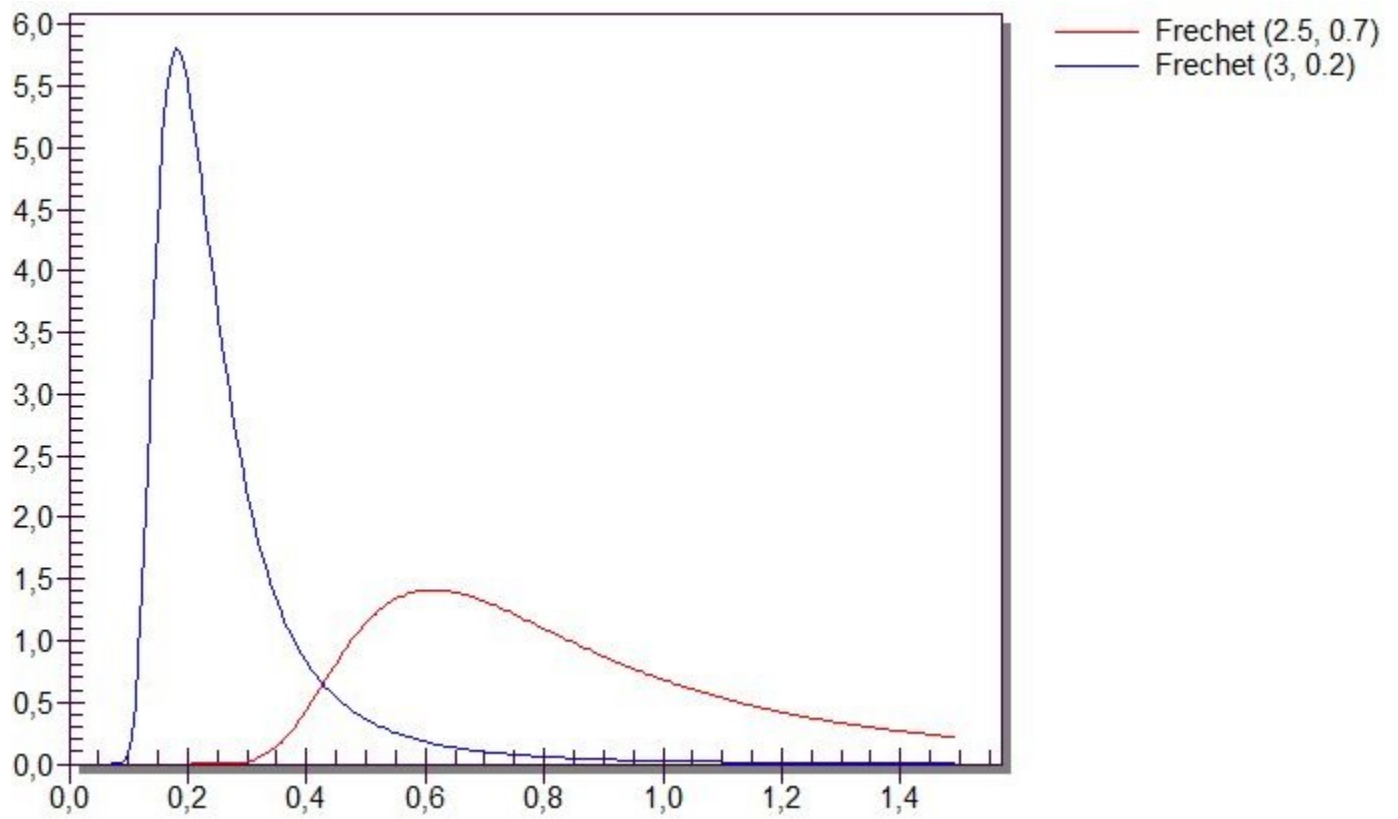
Parameters

The parameters μ and θ have to be positive.

The density function for $x > 0$ is
$$f(x) = \frac{\alpha}{x} \cdot (x/\theta)^{-\alpha} \cdot \exp(-(x/\theta)^{-\alpha})$$

The distribution function is
$$F(x) = \exp(-(x/\theta)^{-\alpha})$$

The following applies $\mu = \theta \cdot \Gamma(1 - 1/\alpha)$ and $\sigma^2 = \theta^2 \cdot \Gamma(1 - 2/\alpha) - \mu^2$ for $\alpha > 2$



Return Value

The return value has the data type *real*.

See also

Parameters of the Distributions

Gumbel [distribution], z_gumbel [SimTalk]

The *Gumbel distribution* is also called the *log-Weibull*, the *Gumbel type I distribution* or the *extreme value distribution*.

Remarks

Use the *Gumbel distribution* to model the distribution of the maximum (or the minimum) of a number of samples of various distributions.

Syntax

```
z_gumbel([Stream:integer,]Mu:time,Theta:real[,LowerBound:real,UpperBound:real]) → real
```

Parameters

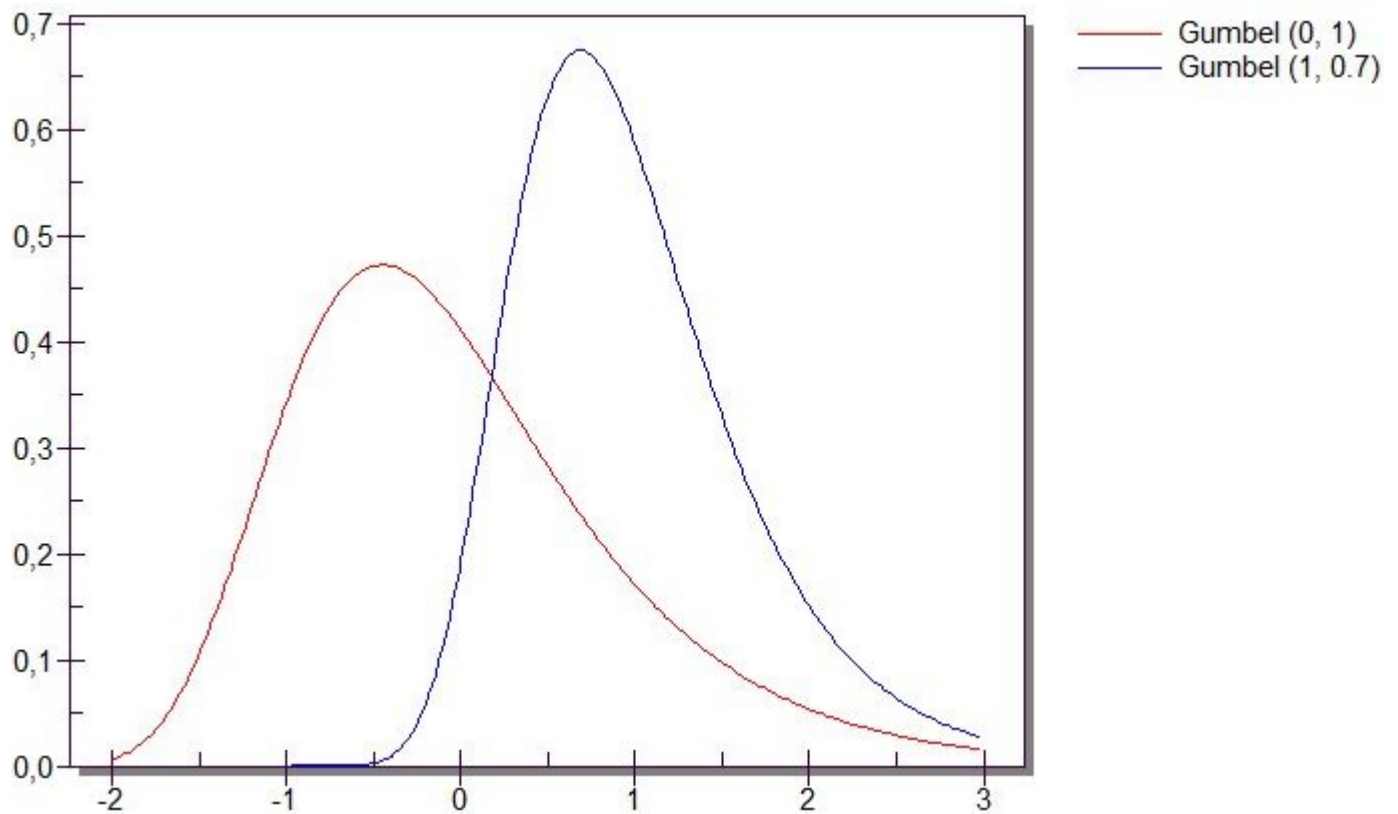
The parameters μ and θ have to be positive.

The parameter μ determines the mean value of the distribution. The variance is $\sigma^2 = (\pi\theta)^2/6$ where $\gamma = 0.57721566\dots$ is the *Euler-Mascheroni constant*.

We set $\mu_0 = \mu - \theta \cdot \gamma$ and $y = \frac{x-\mu_0}{\theta}$

The density function is $f(x) = \frac{1}{\theta} \cdot \exp(-y) \cdot \exp[-\exp(-y)]$

The distribution function is $F(x) = \exp[-\exp(-y)]$



Return Value

The return value has the data type *real*.

See also

Parameters of the Distributions

Laplace [distribution], z_laplace [SimTalk]

The *Laplace distribution* is a continuous distribution. The *Laplace distribution* is also called the *Double-exponential distribution*.

Syntax

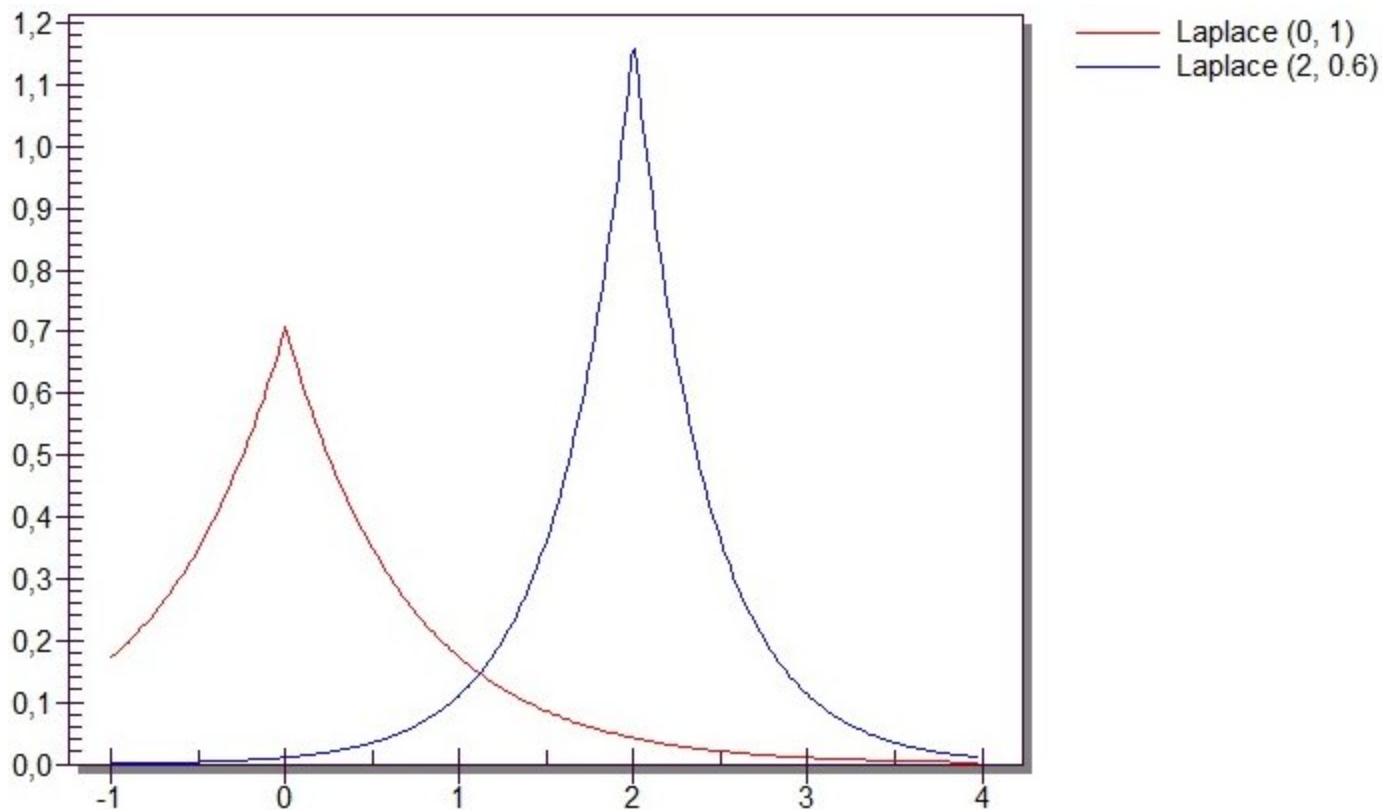
```
z_laplace([Stream:integer,]Mu:time,Theta:real[, LowerBound:real,UpperBound:real]) → real
```

Parameters

The parameters are μ and $\sigma > 0$. The parameter μ sets the mean value of the distribution. The parameter $\sigma > 0$ sets the standard deviation of the distribution.

The density function is $f(x) = \frac{1}{2\beta} \cdot \exp\left(-\frac{|x-\mu|}{\beta}\right)$ where $\beta = \frac{\sigma}{\sqrt{2}}$

For $x < \mu$ the distribution function is $F(x) = \frac{1}{2} \exp\left(\frac{x-\mu}{\beta}\right)$ and for $x \geq \mu$ it is $F(x) = 1 - \frac{1}{2} \exp\left(-\frac{x-\mu}{\beta}\right)$



Return Value

The return value has the data type *real*.

See also

Parameters of the Distributions

Logistic [distribution], z_logistic [SimTalk]

The *logistic distribution* is a continuous probability distribution.

Remarks

Use the *logistic distribution* to model a lifetime distribution.

Syntax

```
z_logistic([Stream:integer,]Mu:time,Theta:real[,LowerBound:real,UpperBound:real]) → real
```

Parameters

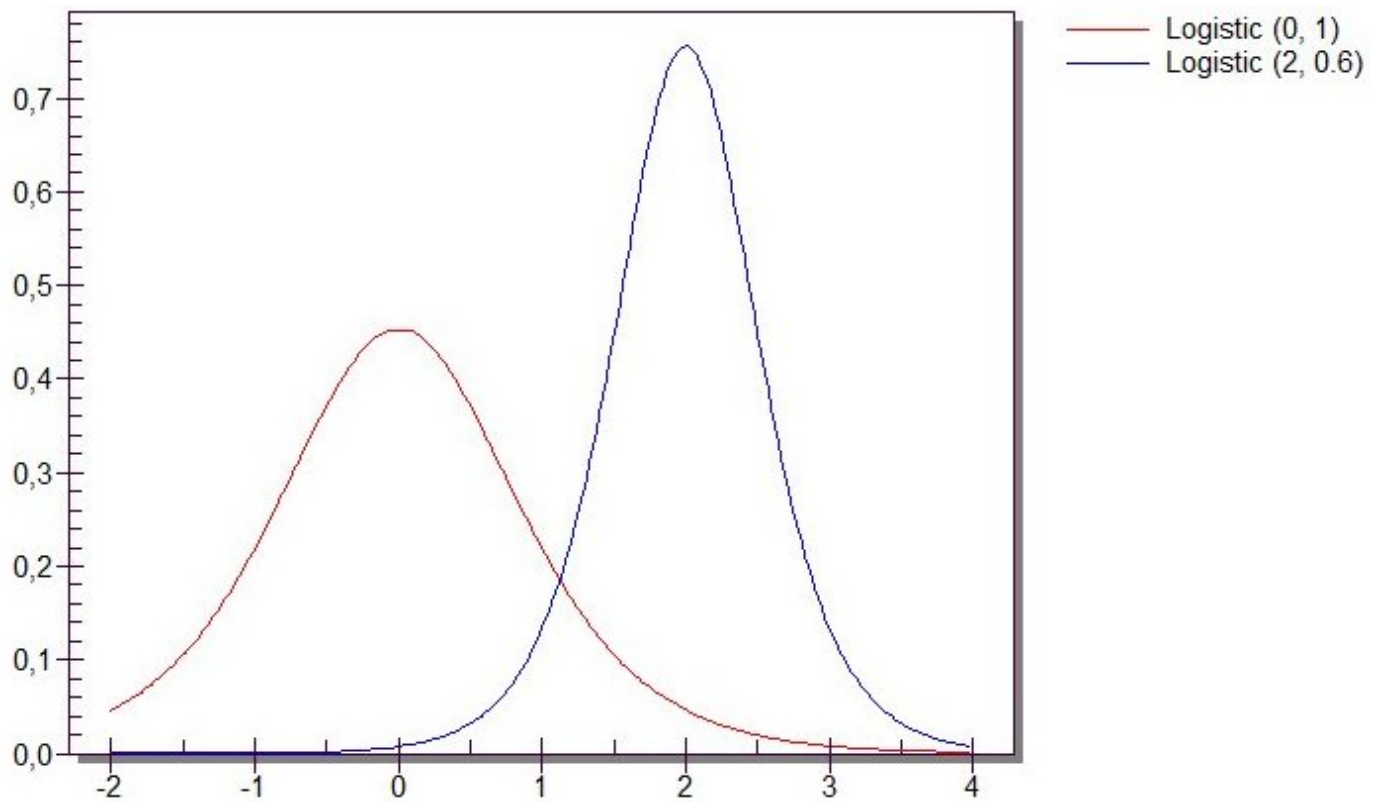
The parameters are μ and $\sigma > 0$.

The parameter μ sets the mean value of the distribution.

The parameter $\sigma > 0$ sets the standard deviation of the distribution.

The density function is
$$f(x) = \frac{\exp(-\frac{x-\mu}{\beta})}{\beta(1 + \exp(-\frac{x-\mu}{\beta}))^2} \text{ where } \beta = \frac{\sigma\sqrt{3}}{\pi}$$

The distribution function is
$$F(x) = \frac{1}{1 + \exp(-\frac{x-\mu}{\beta})}$$



Return Value

The return value has the data type *real*.

See also

Parameters of the Distributions

Loglogistic [distribution], z_loglogistic [SimTalk]

The *loglogistic distribution* is also called the *Fisk distribution*.

Remarks

Use the *loglogistic distribution* to model an income and lifetime distribution.

Syntax

```
z_loglogistic([Stream:integer,]Mu:time,Theta:real[,LowerBound:real,UpperBound:real]) → real
```

Parameters

The parameters are $\alpha > 0$ and $\beta > 0$.

$$f(x; \alpha, \beta) = \frac{(\beta/\alpha)(x/\alpha)^{\beta-1}}{(1 + (x/\alpha)^\beta)^2} \text{ for } x > 0$$

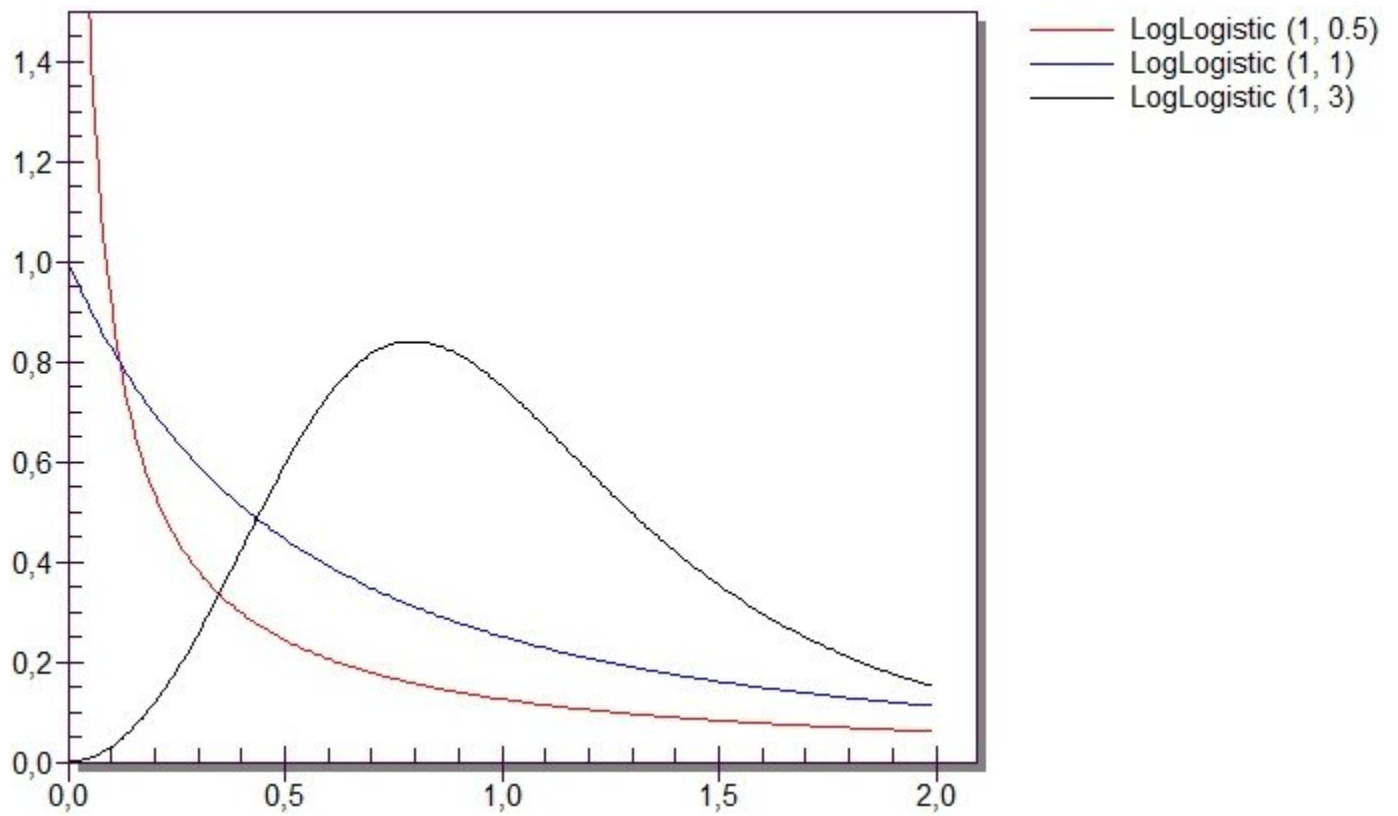
The density function is

$$F(x) = \frac{1}{1 + \left(\frac{x}{\alpha}\right)^{-\beta}}$$

The distribution function is

$$\mu = \frac{\alpha \cdot b}{\sin b} \text{ where } b = \frac{\pi}{\beta}$$

$$\sigma^2 = \alpha^2 \left(\frac{2b}{\sin 2b} - \frac{b^2}{\sin^2 b} \right) \text{ with } \beta > 2$$



Return Value

The return value has the data type *real*.

See also

[Parameters of the Distributions](#)

Paralogistic [distribution], z_paralogistic [SimTalk]

The *paralogistic distribution* is a transformed *Pareto distribution*.

Syntax

```
z_paralogistic([Stream:integer, ]Mu:time,Theta:real[, LowerBound:real,
UpperBound:real]) → real
```

Parameters

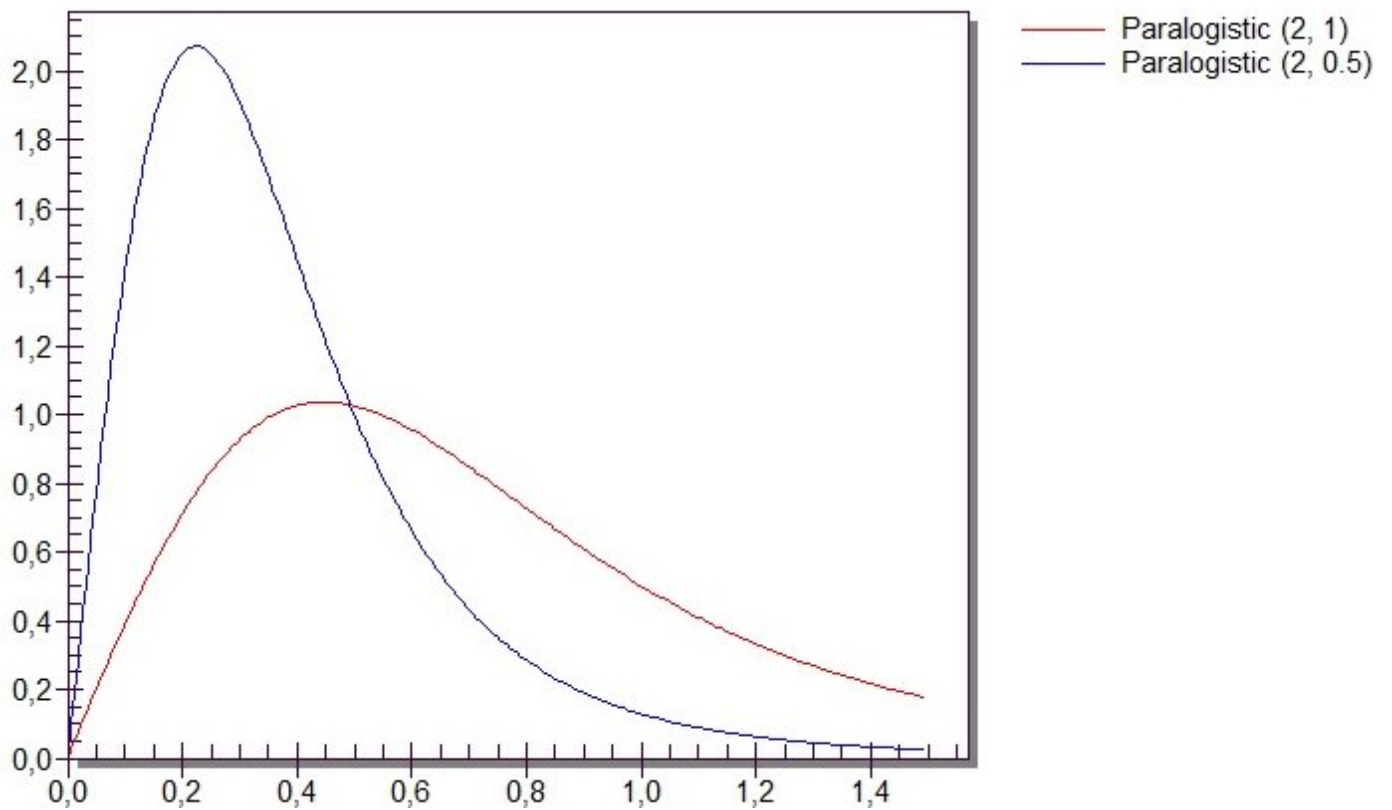
The parameters are $\alpha > 0$ and $\theta > 0$.

The density function is
$$f(x) = \frac{\alpha^2 \cdot (x/\theta)^\alpha}{x \cdot [1 + (x/\theta)^\alpha]^{\alpha+1}}$$
 with $x > 0$.

The distribution function is $F(x) = 1 - h^\alpha$ with $h = \frac{1}{1 + (x/\theta)^\alpha}$

$$\mu = \theta \cdot \Gamma(1 + 1/\alpha) \cdot \Gamma(\alpha - 1/\alpha) / \Gamma(\alpha)$$

$$\sigma^2 = \theta^2 \cdot \Gamma(1 + 2/\alpha) \cdot \Gamma(\alpha - 2/\alpha) / \Gamma(\alpha) - \mu^2 \text{ for } \alpha > \sqrt{2}$$



Return Value

The return value has the data type *real*.

See also

[Parameters of the Distributions](#)

Pareto [distribution], z_pareto [SimTalk]

The *Pareto distribution* is a power-law probability distribution.

Syntax

```
z_pareto([Stream:integer,]Mu:time,Theta:real[,LowerBound:real,UpperBound:real]) → real
```

Parameters

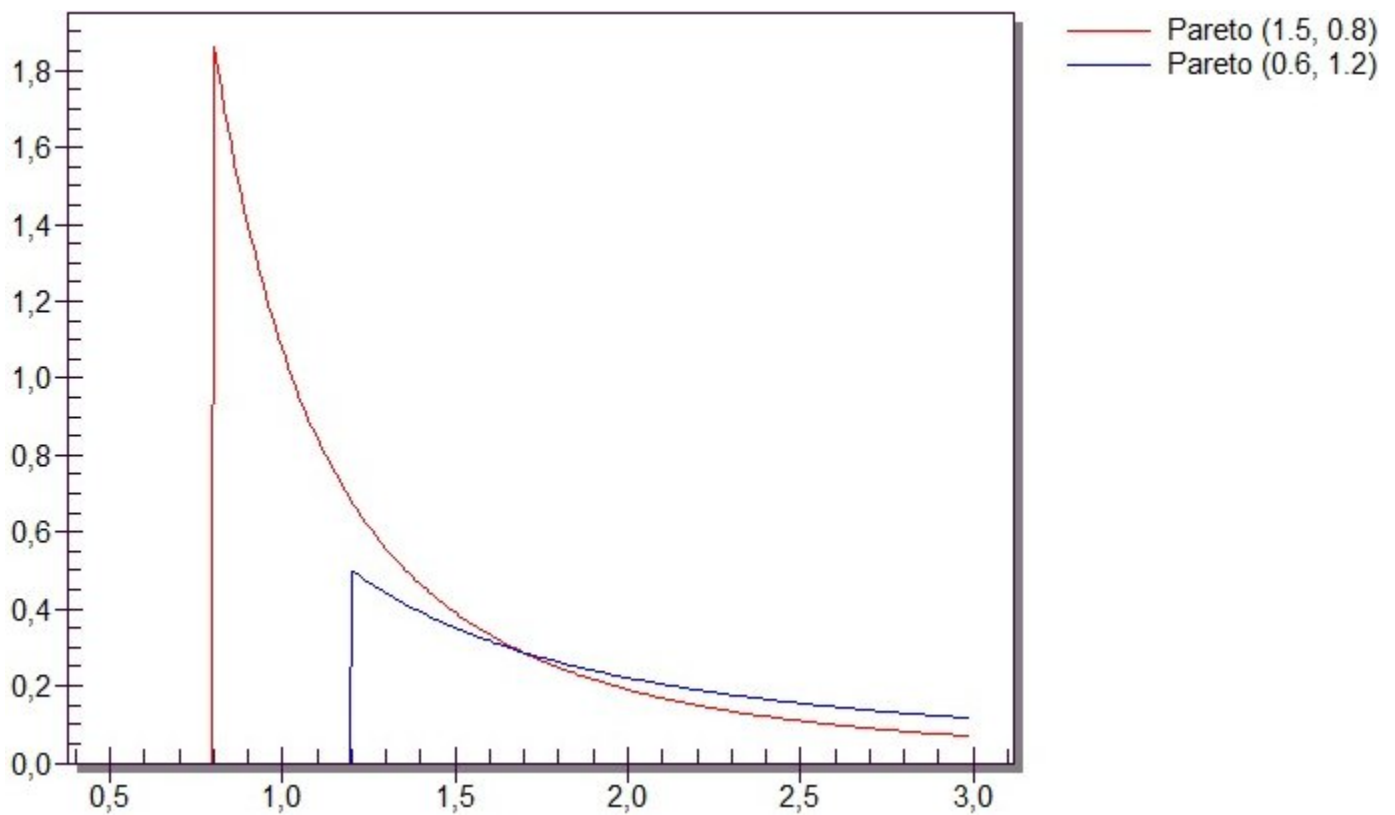
The parameters are $\alpha > 0$ and $\theta > 0$.

The density function is $f(x) = \frac{\alpha \cdot \theta^\alpha}{x^{\alpha+1}} = \frac{\alpha}{\theta} \cdot \left(\frac{\theta}{x}\right)^{\alpha+1}$ for $\alpha > 1$ and $x \geq \theta$

The distribution function is $F(x) = 1 - \left(\frac{\theta}{x}\right)^\alpha$

$$\mu = \frac{\alpha \cdot \theta}{\alpha - 1}$$

$$\sigma^2 = \frac{\alpha \cdot \theta^2}{(\alpha - 1)^2 \cdot (\alpha - 2)} \text{ for } \alpha > 2$$



Return Value

The return value has the data type *real*.

See also

Parameters of the Distributions

Functions for Manipulating Individual Bits

Functions for Manipulating Individual Bits

SimTalk provides the functions listed in the table of contents to the left for manipulating individual bits of an *integer* value.

BitAND [SimTalk]

Executes a bitwise AND of value 1 and value 2 and returns the result.

Type

Function

Syntax

```
BitAND(Value1:integer, Value2:integer) → integer
```

Parameters

You can specify the following parameters:

- The parameter *Value1* of data type *integer* designates value 1.
- The parameter *Value2* of data type *integer* designates value 2.

Return Value

The return value has the data type *integer*.

BitClear [SimTalk]

Changes the value of the parameter *OriginalValue* by setting the bit with the number designated by the parameter *BitPosition* to the bit value 0. The function returns the result of this computation.

Type

Function

Syntax

```
BitClear(OriginalValue:integer, BitPosition:integer) → integer
```

Parameters

You can specify the following parameters:

- The parameter *OriginalValue* of data type *integer* designates the original value of the computation.
- The parameter *BitPosition* of data type *integer* designates the bit position. The value range for the bit position is 0 to 63. The least significant bit has the number 0, the most significant bit has the number 63.

Return Value

The return value has the data type *integer*.

See also

[BitSet \[SimTalk\]](#)

BitOR [SimTalk]

Executes a bitwise OR of value 1 and value 2 and returns the result.

Type

Function

Syntax

```
BitOR(Value1:integer, Value2:integer) → integer
```

Parameters

You can specify the following parameters:

- The parameter *Value1* of data type *integer* designates value 1.
- The parameter *Value2* of data type *integer* designates value 2.

Return Value

The return value has the data type *integer*.

BitSet [SimTalk]

Changes the value of the parameter *OriginalValue* by setting the bit with the number designated by the parameter *BitPosition* to the bit value 1. The function returns the result of this computation.

Type

Function

Syntax

```
BitSet(OriginalValue:integer, BitPosition:integer) → integer
```

Parameters

You can specify the following parameters:

- The parameter *OriginalValue* of data type *integer* designates the original value of the computation.
- The parameter *BitPosition* of data type *integer* designates the bit position. The value range for the bit position is 0 to 63. The least significant bit has the number 0, the most significant bit has the number 63.

Return Value

The return value has the data type *integer*.

See also

[BitClear \[SimTalk\]](#)

BitShift [SimTalk]

Shifts the bits of the value designated by the parameter *integer* using the number of digits designated by *integer* and returns the result.

Type

Function

Syntax

```
BitShift(Value:integer, NumberOfDigits:integer) → integer
```

Parameters

You can specify the following parameters:

- The parameter *Value* of data type *integer* designates the value.
- The parameter *NumberOfDigits* of data type *integer* designates number of digits. It moves them to the left, if you specified digits greater than 0. It moves them to the right, if you specified digits smaller than 0.

Return Value

The return value has the data type *integer*.

BitTest [SimTalk]

Returns if in the *Value* the bit at the position designated by *BitPosition* is set (*true*) or not (*false*).

Type

Function

Syntax

```
BitTest(Value:integer, BitPosition:integer) → boolean
```

Parameters

You can specify the following parameters:

- The parameter *Value* of data type *integer* designates the value.
- The parameter *BitPosition* of data type *integer* designates the bit position. The value range for the bit position is 0 to 63. The least significant bit has the number 0, the most significant bit has the number 63.

Return Value

The return value has the data type *boolean*.

Example

```
var i : integer := 40 // binary representation: 00101000  
print bitTest(i, 3)  // returns true because the bit 3 is set
```

BitXOR [SimTalk]

Executes a bitwise XOR of value 1 and value 2 and returns the result.

Type

Function

Syntax

```
BitXOR(Value1:integer, Value2:integer) → integer
```

Parameters

You can specify the following parameters:

- The parameter *Value1* of data type *integer* designates value 1.
- The parameter *Value2* of data type *integer* designates value 2.

Return Value

The return value has the data type *integer*.

Functions for Manipulating Strings

Functions for Editing Variables of Data Type String

SimTalk provides the functions listed in the table of contents to the left for editing variables of data type *string*.

These functions do not change the contents of the *strings*, but work with copies of the *strings*. The result of this operation is the return value. This enables *Plant Simulation* to enter names of objects into message texts during a simulation run.